Étude expérimentale des algorithmes de tri

Benchmark comparatif — Temps d'exécution selon la taille, le type de tableau et l'algorithme

Algorithmes étudiés :

- Bubble Sort, Selection Sort, Insertion Sort
- Quick Sort, Merge Sort, Heap Sort
- Counting Sort, Bucket Sort

Document de cadrage — Répartition des tâches et protocole expérimental

1. Objectif

L'objectif est d'étudier expérimentalement différents algorithmes de tri afin de comparer leur temps d'exécution selon :

- la taille du tableau `n` ;
- le type de tableau ;
- l'algorithme utilisé.

Chaque membre travaille sur deux algorithmes et doit fournir ses résultats dans **le même format**.

Les résultats seront ensuite regroupés pour réaliser :

- le benchmark global ;
- les graphiques `Temps = f(n)` ;
- l'analyse comparative ;
- le rapport final ;
- la présentation finale.

2. Répartition des tâches

Membre	Algorithmes
Marouane	Bubble Sort + Selection Sort
Asmae	Insertion Sort + Quick Sort
Ayoub	Merge Sort + Heap Sort
Membre 4	Counting Sort + Bucket Sort

Chaque membre est responsable de l'implémentation, des tests et de l'analyse de ses deux algorithmes.

3. Livrable de chaque membre

Chaque membre doit fournir **un seul dossier** contenant les éléments suivants pour ses deux algorithmes.

```
Nom_Membre/
|
|- source/
|- algorithme_1.c
|- algorithme_2.c
|- tests/
|- resultats
```

```
|- analyse/
|- analyse.pdf ou .md
|- presentation/
|- presentation.pdf
```

4. Source code C

Pour chaque algorithme :

- code en **C** ;
- code propre et commenté ;
- fonction de tri clairement identifiable ;
- pas de code inutile ;
- même principe d'entrée pour faciliter le benchmark global.

Exemple :

```
void bubbleSort(int arr[], int n);
```

Les deux algorithmes doivent pouvoir être exécutés avec différentes tailles `n`.

5. Tests à réaliser

Chaque algorithme doit être testé sur plusieurs types de tableaux.

Cas 1 — Tableau aléatoire

```
[8, 3, 9, 1, 5, 2, 7, 4, 6]
```

Cas 2 — Tableau déjà trié

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Cas 3 — Tableau décroissant

```
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Cas 4 — Tableau presque trié

```
[1, 2, 3, 5, 4, 6, 7, 9, 8]
```

6. Tailles des tableaux

Pour commencer, utiliser les mêmes tailles pour tous les membres :

Tailles n à utiliser			
100	500	1 000	5 000
10 000	20 000	50 000	100 000

Cependant, si un algorithme devient trop lent, il est possible de réduire les grandes tailles pour les algorithmes `O(n²)`.

Dans ce cas, il faut **indiquer clairement les tailles réellement utilisées**.

7. Mesure du temps

Le temps d'exécution doit être mesuré en C avec une méthode commune, par exemple `clock()` de `<time.h>`.

Pour chaque expérience :

```
Algorithme
+
Type de tableau
+
Taille n
+
Temps d'execution
```

Il est recommandé de répéter chaque test plusieurs fois et de calculer une moyenne afin d'obtenir des résultats plus stables.

Exemple de données à fournir

n	Temps moyen
Algorithme : Bubble Sort — Type : Random	
100	...
500	...
1 000	...
5 000	...
10 000	...

8. Format des résultats pour le benchmark global

Très important : tous les membres doivent utiliser le même format.

Pour chaque algorithme, fournir les données sous forme de tableau ou CSV :

```
algorithm,type,n,time
bubble,random,100,...
bubble,random,500,...
bubble,random,1000,...
bubble,sorted,100,...
...
```

Les types seront :

Types de tableaux			
random	sorted	reverse	nearly_sorted

Le temps doit être dans **la même unité pour tout le monde**.

Par exemple :

```
milliseconds (ms)
```

L'objectif est que les résultats puissent ensuite être regroupés automatiquement pour générer les graphiques.

9. Analyse de chaque algorithme

Pour chaque algorithme, donner une analyse courte comprenant :

Principe

Quelques lignes expliquant comment fonctionne l'algorithme.

Complexité

Indiquer :

- meilleur cas ;
- cas moyen ;
- pire cas ;
- complexité mémoire.

Résultats observés

Expliquer brièvement :

- quel type de tableau semble favorable ;
- quel type semble défavorable ;
- comment le temps évolue avec `n`.

Comparaison théorie / expérience

Dire si les résultats expérimentaux correspondent globalement à la complexité théorique.

L'analyse doit rester **courte et basée sur les résultats obtenus**.

10. Petite présentation de chaque membre

Chaque membre prépare une présentation de **maximum 2 pages/slides** pour ses deux algorithmes.

Pas de longs textes.

Page 1	Page 2
Algorithme 1	Algorithme 2
- Principe - Complexité - Meilleur cas - Pire cas - Résultat principal	- Principe - Complexité - Meilleur cas - Pire cas - Résultat principal

Ajouter éventuellement **un petit graphique ou tableau** si les résultats sont déjà disponibles.

La présentation globale sera réalisée plus tard à partir de ces éléments.

11. Ce qui doit être remis

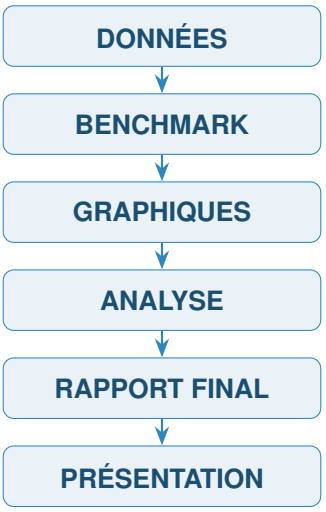
Chaque membre doit donc fournir :

Obligatoire

- ☐ Source code C des 2 algorithmes
- ☐ Tests de fonctionnement
- ☐ Tests de performance
- ☐ Résultats des temps d'exécution
- ☐ Résultats dans le format commun
- ☐ Analyse courte des 2 algorithmes
- ☐ Présentation de maximum 2 pages

Objectif

Le livrable doit permettre à l'équipe de récupérer directement les résultats pour construire ensuite :



12. Important

Pour que les résultats soient comparables :

- même langage : **C** ;
- même méthode de mesure du temps ;
- même unité de temps ;
- mêmes types de tableaux ;
- mêmes tailles `n` lorsque possible ;
- mêmes conditions de test ;
- chaque algorithme doit produire un tableau correctement trié avant de considérer le résultat valide.

Ne pas modifier les résultats pour qu'ils correspondent à la théorie.

Les mesures obtenues doivent être conservées telles quelles, puis analysées.

Résultat attendu

À la fin de cette première étape, nous devons avoir **4 livrables individuels homogènes**.

Ils serviront ensuite à créer un **benchmark global unique**, avec les données de :

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Merge Sort
- Heap Sort

- Counting Sort
- Bucket Sort

Les graphiques et le rapport final seront réalisés **après avoir regroupé et vérifié tous les résultats**.